

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

«КИЇВСЬКИЙ АВІАЦІЙНИЙ ІНСТИТУТ»

Факультет комп'ютерних наук та технологій

Кафедра прикладної математики

КУРСОВА РОБОТА

з навчальної дисципліни «Методи оптимізацій та дослідження операцій»

**Тема: «Теоретико-ігрові моделі в генеративно-змагальних нейромережах
(GANs)»**

Виконав студент групи Б-122-23-1-ШІ

Щербинський Дмитро Геннадійович

Керівник: кандидат фіз-мат наук, доцент Хребет В.Г.

КИЇВ 2026

ЗМІСТ

1 Вступ	3
2 Теоретична частина	5
2.1 Теоретико-ігрова основа генеративно-змагальних мереж	5
2.2 Класична архітектура GAN	7
2.3 Згорткова архітектура DCGAN	9
2.4 Wasserstein GAN та метрична основа збіжності	11
2.5 Hinge Loss GAN	15
2.6 Геометрія гри та поля градієнтів	16
2.7 Узагальнена теоретико-ігрова модель GAN	18
3 Практична частина	21
3.1 Програмна реалізація експериментального пайплайну	21
3.2 Серія експериментів: Vanilla GAN (BCE-loss)	23
3.3 Серія експериментів: Wasserstein GAN	25
3.4 Серія експериментів: Hinge Loss GAN	28
3.5 Підсумкові результати генерації	32
4 Висновки	33
5 Список використаних джерел	34
6 Додатки	35

1. ВСТУП

Актуальність теми. Генеративно-змагальні нейромережі (GAN) є одним із актуальних напрямів сучасного машинного навчання з моменту їх введення у роботі Гудфелоу та співавторів [1]. Архітектура GAN формалізує процес навчання як гру двох гравців: генератора, що генерує штучні зразки, та дискримінатора, що відрізняє їх від реальних. Така постановка природно вписується в апарат теорії ігор, зокрема в концепцію рівноваги Неша та мінімакських стратегій [2]. Попри практичні успіхи, теоретичні засади збіжності й існування рівноваги в GAN залишаються предметом активних досліджень [3]: навчання є нестабільним, а рівновага досягається лише за певних умов, що вимагає як теоретичного аналізу, так і вдосконалення технік тренування [4]. Таким чином, вивчення теоретико-ігрових моделей у контексті GAN є актуальною науковою задачею на перетині математичної теорії ігор та глибокого навчання.

Об’єкт дослідження – процес навчання генеративно-змагальних нейромереж як динамічна гра двох гравців.

Предмет дослідження – математичні умови існування та досягнення рівноваги Неша в мінімакській задачі GAN, а також методи стабілізації процесу навчання.

Мета роботи. Метою роботи є теоретичний аналіз та практична перевірка теоретико-ігрових властивостей GAN на задачі генерації зображень котятих морд із датасету Animal Faces [5].

Завдання роботи:

1. формалізувати гру двох гравців (генератора та дискримінатора) у термінах теорії ігор;
2. дослідити умови існування рівноваги Неша та її зв'язок із мінімаксною задачею;
3. розглянути альтернативні формулювання цільової функції (зокрема Wasserstein GAN [6]) з точки зору покращення збіжності;
4. реалізувати архітектуру DCGAN [7] для генерації зображень;
5. експериментально дослідити поведінку моделі на датасеті Animal Faces та проаналізувати отримані результати.

Методи дослідження, технічні та програмні засоби. У роботі використано методи теорії ігор (аналіз мінімакських стратегій, рівновага Неша), методи математичного аналізу (варіаційні відстані між розподілами, відстань Васерштайна), а також методи глибокого навчання (згорткові нейромережі, зворотне поширення помилки). Технічні засоби: мова програмування Python 3, бібліотеки PyTorch, NumPy, Matplotlib. Обчислення виконуються на платформі Kaggle.

Практичне значення одержаних результатів. Результати дослідження дозволяють краще зрозуміти умови стабільного навчання GAN та можуть бути застосовані при проектуванні генеративних моделей для синтезу зображень, аугментації даних і суміжних задач комп'ютерного зору.

2. ТЕОРЕТИЧНА ЧАСТИНА

2.1. Теоретико-ігрова основа генеративно-змагальних мереж

Теорія ігор вивчає математичні моделі стратегічної взаємодії між раціональними агентами. Центральним поняттям є **рівновага Неша** – профіль стратегій, за якого жоден гравець не може покращити свій виграш одностороннім відхиленням [2]. У контексті GAN ця концепція набуває конкретного операційного змісту: навчання мережі є не задачею мінімізації єдиного функціонала, а пошуком сідлової точки цільової функції двох гравців.

2.1.1. Формальна постановка гри

Нехай $\mathcal{X}$ – простір реальних даних із невідомим розподілом p_{data} . Генератор $G : \mathcal{Z} \rightarrow \mathcal{X}$ відображає латентний вектор $z \sim p_z$ у простір даних, індукуючи розподіл p_g . Дискримінатор $D : \mathcal{X} \rightarrow [0, 1]$ повертає ймовірність того, що поданий зразок належить p_{data} , а не p_g .

Гра визначається **функцією цінності** [1]:

$$V(G, D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Дискримінатор максимізує V , генератор – мінімізує, тому задача записується як:

$$\min_G \max_D V(G, D)$$

Це і є **мінімаксна гра з нульовою сумою** у нормальній формі.

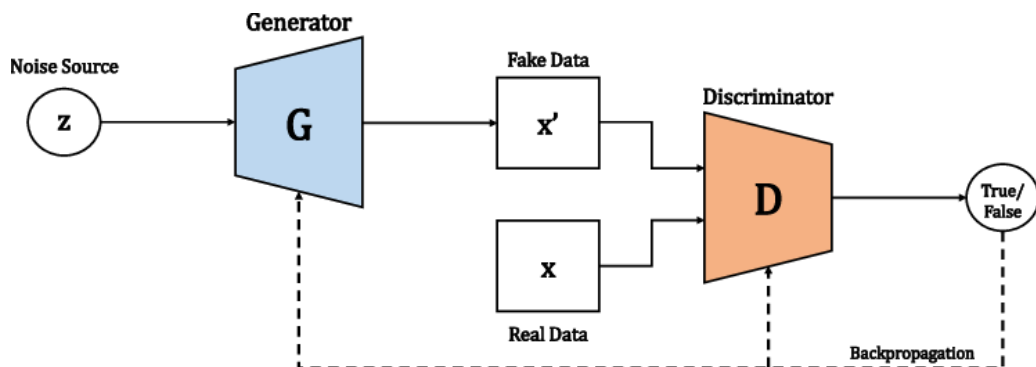


Рисунок 1: Схема взаємодії генератора та дискримінатора

2.1.2. Оптимальний дискримінатор та зв'язок з дивергенцією Єнсена–Шеннона

При фіксованому G оптимальний дискримінатор знаходиться аналітично. Для довільного x підінтегральний вираз $a \log y + b \log(1 - y)$ максимізується при $y = \frac{a}{a+b}$, звідси [1]:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

Підставляючи D_G^* у V , отримуємо критерій для генератора:

$$C(G) = \max_D V(G, D) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \parallel p_g)$$

де JSD – симетрична дивергенція Єнсена–Шеннона. Оскільки $\text{JSD} \geq 0$ і $\text{JSD} = 0 \iff p_{\text{data}} = p_g$, глобальний мінімум $C(G) = -\log 4$ досягається тоді й тільки тоді, коли $p_g = p_{\text{data}}$ [1].

2.1.3. Існування та єдиність рівноваги Неша

Теорема (Гудфелоу та ін., 2014). Якщо G і D мають достатню ємність, то єдиною точкою рівноваги є $p_g = p_{\text{data}}$, $D^*(x) = \frac{1}{2}$ для всіх x [1].

Однак для нескінченних ігор з неперервними стратегіями існування рівноваги у чистих стратегіях не гарантоване в загальному випадку. Голцман [2] показав, що у нескінченних двоосібних іграх з нульовою сумою мінімаксна рівність $\min_G \max_D = \max_D \min_G$ виконується лише за додаткових умов компактності або опуклості-угнутості. Для GAN ці умови порушуються, бо V є неопуклою по параметрах G (нейромережа), що пояснює практичну нестабільність навчання.

2.1.4. Аналіз стійкості змагальної динаміки

Розглянемо градієнтну динаміку одночасного оновлення параметрів:

$$\theta_D \leftarrow \theta_D + \eta \nabla_{\theta_D} V, \quad \theta_G \leftarrow \theta_G - \eta \nabla_{\theta_G} V$$

Лінеаризуючи навколо точки рівноваги θ^* , отримуємо систему:

$$\dot{\theta} = F(\theta), \quad F = \begin{pmatrix} \nabla_{DD}^2 V & \nabla_{DG}^2 V \\ -\nabla_{GD}^2 V & -\nabla_{GG}^2 V \end{pmatrix}$$

Стійкість визначається власними значеннями якобіана F . Аура та Чжан [3] довели, що навіть якщо рівновага існує, градієнтна динаміка може не збігатися до неї через наявність уявних власних значень – система «обертається» навколо рівноваги, а не сходиться до неї. Саліманс та ін. [4] запропонували практичні прийоми стабілізації: **feature matching** (мінімізація відхилення між статистиками проміжних шарів дискримінатора), **minibatch discrimination** (облік різноманітності всередині батчу) та **historical averaging** (регуляризація поточних параметрів відносно їх середнього по часу).

2.2. Класична архітектура GAN

2.2.1. Архітектура та алгоритм навчання

Оригінальна архітектура GAN [1] складається з двох повнозв'язних мереж. Генератор $G(z; \theta_g)$ приймає вектор шуму $z \in \mathbb{R}^k$ (зазвичай $z \sim \mathcal{N}(0, I)$) і породжує зразок у просторі даних. Дискримінатор $D(x; \theta_d)$ повертає скалярну оцінку автентичності.

Навчання чергує два кроки на кожній ітерації (а та b) [1]:

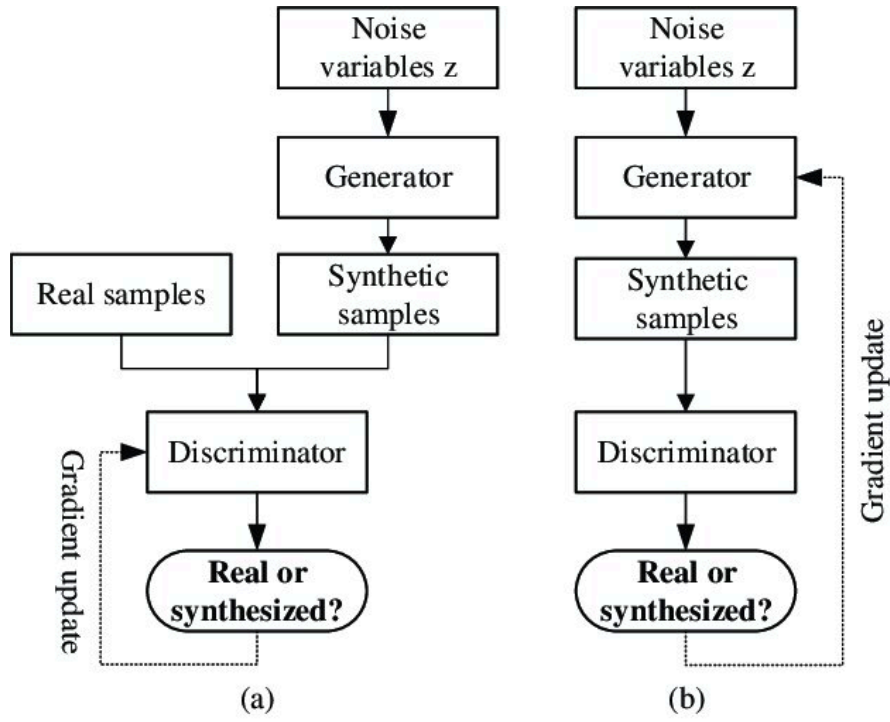


Рисунок 2: Алгоритм почергового оновлення дискримінатора та генератора

Крок а (оновлення D, k разів):

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m [\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)})))]$$

Крок б (оновлення G):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(\mathbf{z}^{(i)})))$$

На практиці другий крок замінюють на максимізацію $\log D(G(\mathbf{z}))$ («несатуруюча» цільова функція), щоб уникнути насичення градієнтів на початку навчання, коли дискримінатор легко відрізняє фейкові зразки [1].

2.2.2. Колапс мод та проблема збіжності

Ключова практична проблема класичного GAN – **mode collapse**: генератор вироджується до відтворення малої кількості зразків, що обманюють дискримінатор, замість охоплення повного розподілу p_{data} . З теоретичного боку Арора та Чжан [3] показали, що стандартний дискримінатор кінцевої ємності не здатен розрізнити розподіл, що покриває усі моди, від розподілу, що покриває лише їх

поліноміальну підмножину, – отже, сигнал градієнта виявляється недостатнім для уникнення колапсу.

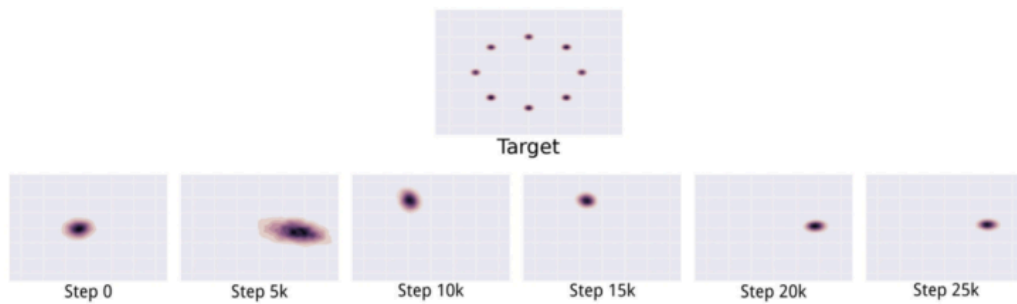


Рисунок 3: Ілюстрація mode collapse: генератор відтворює лише частину мод розподілу

2.3. Згорткова архітектура DCGAN

2.3.1. Мотивація та ключові рішення

Радфорд та ін. [7] запропонували **Deep Convolutional GAN (DCGAN)** – перенесення принципів GAN на згорткові архітектури для роботи із зображеннями. Основні архітектурні рішення:

- Заміна субдискретизації (pooling) на **stride-згортки** у дискримінаторі та **дробові stride-згортки** (транспоновані згортки) у генераторі.
- **Batch Normalization** після кожного шару (крім вихідного шару генератора та вхідного дискримінатора) – стабілізує градієнти та прискорює збіжність.
- Відмова від повнозв’язних шарів усередині мережі.
- **ReLU**-активація у генераторі (з $\tanh$ на виході), **LeakyReLU** у дискримінаторі.

2.3.2. Архітектура генератора

Вхідний вектор $z \in \mathbb{R}^{\{100\}}$ проєктується у тензор $4 \times 4 \times 512$ за допомогою лінійного шару та серії транспонованих згорток зі збільшенням просторового розміру вдвічі на кожному кроці:

$$\mathbb{R}^{\{100\}} \rightarrow 4 \times 4 \times 1024 \rightarrow 8 \times 8 \times 512 \rightarrow 16 \times 16 \times 256 \rightarrow 32 \times 32 \times 128 \rightarrow 64 \times 64 \times 3$$

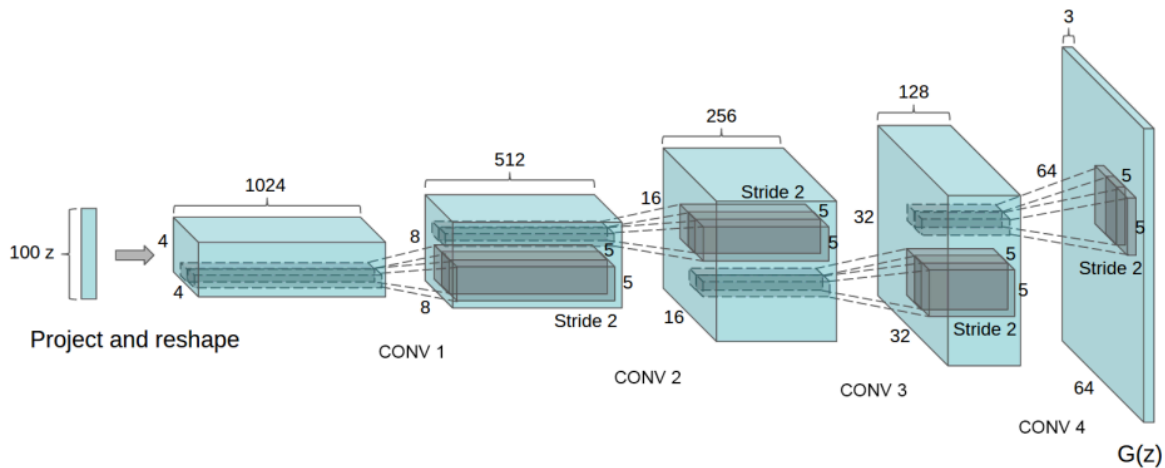


Рисунок 4: Архітектура генератора DCGAN: послідовне підвищення просторового розміру

2.3.3. Архітектура дискримінатора

Дискримінатор є дзеркальною до генератора згортковою мережею: кожен шар зменшує просторовий розмір удвічі та збільшує кількість каналів:

$$64 \times 64 \times 3 \rightarrow 32 \times 32 \times 128 \rightarrow 16 \times 16 \times 256 \rightarrow 8 \times 8 \times 512 \rightarrow 4 \times 4 \times 1024 \rightarrow 1$$

LeakyReLU (нахил 0.2 для від'ємних значень) дозволяє градієнтам проходити навіть при слабкій активації, що є критично важливим для стабільного навчання дискримінатора [7].

2.3.4. Внесок просторової узгодженості

Ванг та ін. [8] виявили, що класичний дискримінатор DCGAN оцінює зображення **глобально**, ігноруючи локальну просторову структуру, що призводить до артефактів. Запропоноване рішення – **просторово усвідомлений дискримінатор** (spatial-aware discriminator), що вирівнює карти ознак за координатними осями перед прийняттям рішення. Це підвищує рівновагу між G та D, оскільки дискримінатор отримує більш інформативний сигнал про локальні невідповідності.

2.4. Wasserstein GAN та метрична основа збіжності

2.4.1. Проблема дивергенції Єнсена–Шеннона

Як показано в розділі 2.1, навчання класичного GAN еквівалентне мінімізації $JSD(p_{\text{data}} \parallel p_g)$. Однак JSD має критичний недолік: якщо носії p_{data} та p_g не перетинаються (що типово на початку навчання, коли p_g є «тонким» многовидом у просторі зображень), $JSD = \log 2$ є константою і **не несе жодного градієнтного сигналу** [6].

Аналогічна проблема виникає з дивергенцією Kullback–Leibler: $KL(p_g \parallel p_{\text{data}}) = +\infty$, якщо p_g має масу там, де $p_{\text{data}} = 0$. У цих випадках градієнт зникає або вибухає, що й спричиняє нестабільність.

2.4.2. Відстань Васерштайна

Арйовскі, Чінтала та Боттоу [6] запропонували замінити JSD на **відстань Васерштайна першого порядку** (Earth Mover’s Distance):

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{\tilde{x} \sim p_g} [f(\tilde{x})]$$

де супремум береться по всіх 1-ліпшицевих функціях f . Ключова перевага: W є неперервною та диференційовною всюди (навіть коли носії розподілів не перетинаються), забезпечуючи стабільний градієнтний сигнал протягом усього навчання.

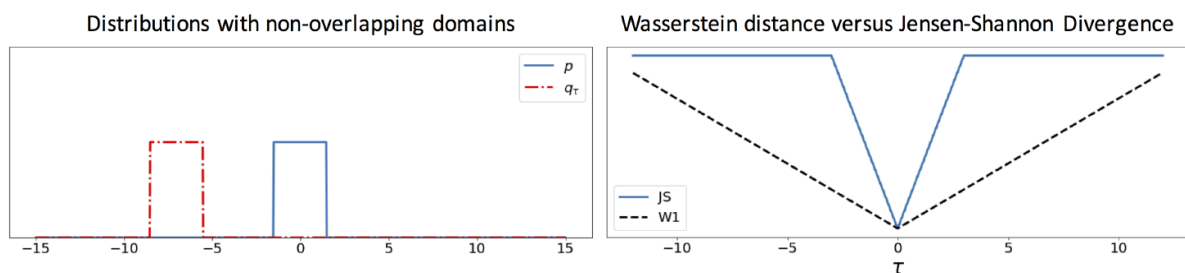


Рисунок 5: Порівняння JSD та відстані Васерштайна: JSD насичується при непересічних носіях, W залишається інформативною

2.4.3. Критик та умова Ліпшиця

У WGAN дискримінатор перейменовується на **критика** $f_w : \mathcal{X} \rightarrow \mathbb{R}$ (без сигмоїди на виході) і максимізує:

$$L = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [f_w(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p_z} [f_w(G(\mathbf{z}))]$$

Генератор мінімізує $-\mathbb{E}[f_w(G(\mathbf{z}))]$. Умова 1-ліпшицевості $\|\nabla f_w\| \leq 1$ накладається через **weight clipping** (обрізання ваг до $[-c, c]$) у оригінальній роботі [6], що є грубим наближенням.

Мінімаксна задача набуває вигляду:

$$\min_G \max_{w: \|f_w\|_L \leq 1} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [f_w(\mathbf{x})] - \mathbb{E}_{\mathbf{z} \sim p_z} [f_w(G(\mathbf{z}))]$$

2.4.4. WGAN з штрафом на градієнт (WGAN-GP)

Практика показала, що weight clipping у оригінальному WGAN породжує власні проблеми. По-перше, обрізання ваг змушує критика використовувати лише малий діапазон параметрів, що призводить до **деградації ємності** мережі та надмірно простих функцій f_w . По-друге, при занадто малому порозі c градієнти зникають, при занадто великому – вибухають, що ускладнює підбір гіперпараметра. Гулраджані та ін. [9] запропонували замінити пряме обмеження на м'який штраф, що отримав назву **gradient penalty** (GP).

Теорема ([9]). Нехай π – оптимальний транспортний план між p_{data} та p_g . Тоді оптимальний критик f^* має одиничну норму градієнта майже всюди вздовж прямих, що з'єднують точки з носіїв p_{data} та p_g :

$$\mathbb{E}_{(\mathbf{x}, \tilde{\mathbf{x}}) \sim \pi} [\|\nabla f^*(\mathbf{x}_t)\|_2] = 1, \quad \mathbf{x}_t = t\mathbf{x} + (1-t)\tilde{\mathbf{x}}, \quad t \sim U[0, 1]$$

Замість того щоб гарантувати $\|\nabla f_w\| \leq 1$ обрізанням, WGAN-GP безпосередньо штрафує відхилення норми градієнта від одиниці на інтерпольованих точках. Цільова функція критика набуває вигляду:

$$L_{GP} = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [f_w(x)] - \mathbb{E}_{z \sim p_z} [f_w(G(z))]}_{\text{оцінка Васерштайна}} - \underbrace{\lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} [(\|\nabla_{\hat{x}} f_w(\hat{x})\|_2 - 1)^2]}_{\text{штраф на градієнт}}$$

де $\hat{x} = \varepsilon x + (1 - \varepsilon)G(z)$, $\varepsilon \sim U[0, 1]$, а λ – коефіцієнт штрафу (типово $\lambda = 10$).

Оновлення параметрів генератора залишається незмінним:

$$\min_{\theta_G} -\mathbb{E}_{z \sim p_z} [f_w(G(z))]$$

Суттєвою відмінністю від WGAN є те, що WGAN-GP **не потребує** batch normalization у критику, оскільки нормалізація по батчу порушує незалежність між зразками й унеможливлює коректне обчислення градієнтного штрафу. Натомість використовують **layer normalization** або взагалі відмовляються від нормалізації в критику.

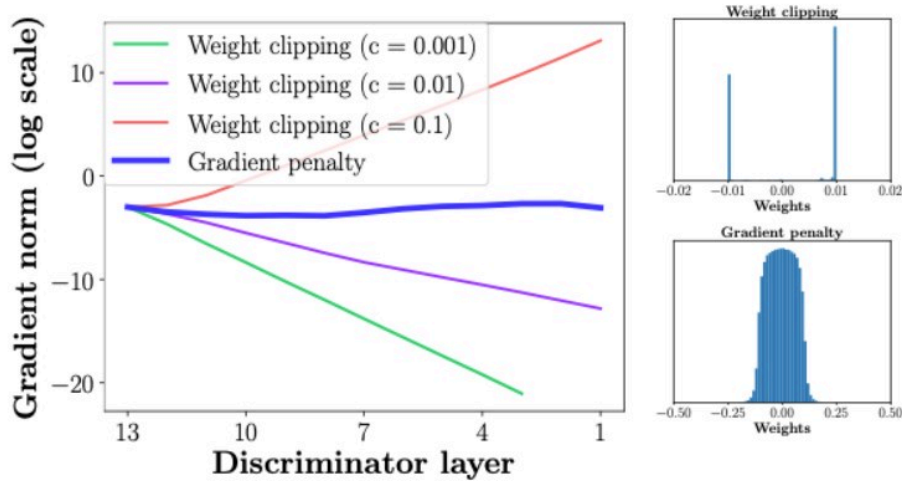


Рисунок 6: Розподіл норми градієнта критика: weight clipping (ліворуч) концентрує значення на 0 та c ; gradient penalty (праворуч) утримує норму поблизу 1. З точки зору теорії ігор gradient penalty є **регуляризацією простору стратегій** критика: замість жорсткого обмеження множини допустимих стратегій $\{f : \|f\|_L \leq 1\}$ вводиться м'яке пенальті, що зміщує рівновагу до функцій із потрібними властивостями. Це еквівалентно переходу від констрейнованої до **пенальтизованої** мінімаксної задачі:

$$\min_G \max_w \left[\mathbb{E} [f_w(x)] - \mathbb{E} [f_w(G(z))] - \lambda \mathbb{E} [(\|\nabla f_w(\hat{x})\|_2 - 1)^2] \right]$$

Гулраджані та ін. показали, що WGAN-GP збігається стабільніше та швидше за WGAN з weight clipping на широкому класі архітектур, включаючи ResNet-подібні генератори.

2.4.5. Теоретико-ігрова інтерпретація

З позицій теорії ігор WGAN та WGAN-GP замінюють гру з нульовою сумою відносно JSD на гру відносно метрики Васерштайна. Це суттєво: тоді як JSD є лише псевдометрикою (не задовольняє умові неперервності на непересічних носіях), W є справжньою метрикою на просторі розподілів, тому поверхня гри є «гладшою» і умови збіжності до рівноваги виконуються за ширшого класу конфігурацій [6]. WGAN-GP додатково покращує властивості поверхні, забезпечуючи регулярність критика та усуваючи патологічні локальні мінімакси, що виникають при weight clipping.

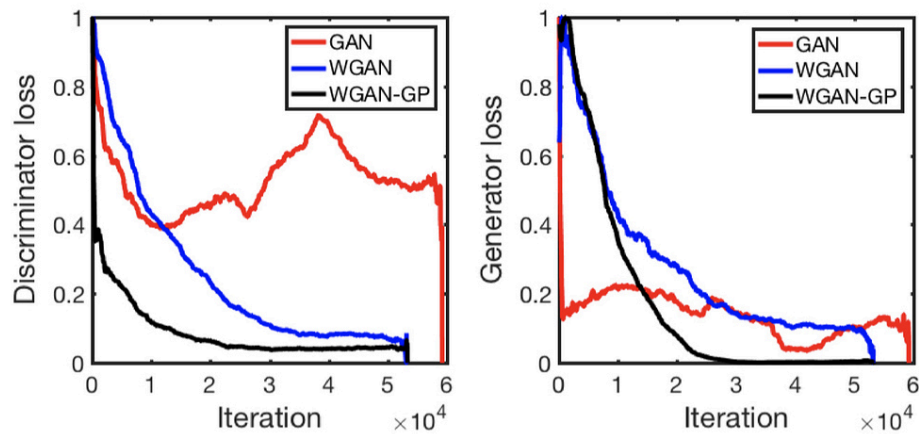


Рисунок 7: Стабільність навчання WGAN та WGAN-GP порівняно з класичним GAN

2.5. Hinge Loss GAN

2.5.1. Формулювання

Hinge loss, запозичена з теорії опорних векторів, визначає цільові функції через відступ (margin) шириною 2 навколо нуля [10]:

$$L_D = -\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\min(0, -1 + D(\mathbf{x}))] - \mathbb{E}_{\mathbf{z} \sim p_z} [\min(0, -1 - D(G(\mathbf{z})))]$$

$$L_G = -\mathbb{E}_{\mathbf{z} \sim p_z} [D(G(\mathbf{z}))]$$

Дискримінатор прагне до $D(\mathbf{x}) \geq 1$ для реальних зразків і $D(G(\mathbf{z})) \leq -1$ для синтетичних. Генератор максимізує скалярну оцінку дискримінатора без логарифмічного насичення. На відміну від BCE, штраф нараховується лише у перехідній зоні $D(\mathbf{x}) \in (-1, +1)$; поза нею градієнт дорівнює нулю:

$$D(\mathbf{x}) \begin{cases} \geq +1 & \mathbf{x} \sim p_{\text{data}}, \text{ градієнт} = 0 \\ \in (-1, +1) & \text{перехідна зона, градієнт сталий} \\ \leq -1 & \mathbf{x} \sim p_g, \text{ градієнт} = 0 \end{cases}$$

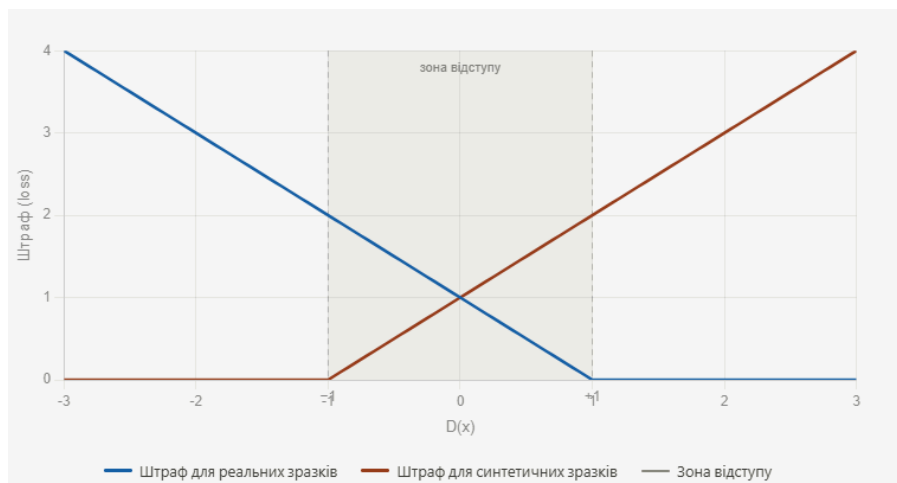


Рисунок 8: Hinge loss: штраф нараховується лише у перехідній зоні; поза відступом градієнт нульовий

2.5.2. Зв'язок зі spectral normalization

Hinge loss зазвичай застосовується разом зі **spectral normalization** (SN) [10], яка нормує кожну матрицю ваг дискримінатора за спектральною нормою:

$$W_{\text{SN}} = \frac{W}{\sigma(W)}$$

Це гарантує 1-ліпшицевість без weight clipping і без додаткового штрафного доданку. Комбінація SN і hinge loss утворює архітектуру SNGAN.

2.5.3. Теоретико-ігрова інтерпретація

Рівновага Неша для hinge loss відповідає стану $D(x) = 1$, $D(G(z)) = -1$: жоден гравець не має стимулу змінювати стратегію, не виходячи за межі відступу. На відміну від BCE, де дискримінатор отримує ненульовий градієнт навіть при впевненій класифікації, hinge loss припиняє оновлення дискримінатора після досягнення відступу, що природно обмежує «гонку» між G та D.

2.6. Геометрія гри та поля градієнтів

2.6.1. Поле градієнтів та сідлова точка

Мінімаксну задачу можна описати через **поле градієнтів** одночасного оновлення параметрів:

$$v(\theta) = \begin{pmatrix} -\nabla_{\theta_G} \mathcal{L}(\theta_G, \theta_D) \\ \nabla_{\theta_D} \mathcal{L}(\theta_G, \theta_D) \end{pmatrix}$$

Сідлова точка θ^* – нуль цього поля: $v(\theta^*) = 0$. На відміну від задачі мінімізації, де нульовий градієнт гарантує критичну точку, у мінімаксній грі $v(\theta^*) = 0$ може відповідати обертанню навколо рівноваги без збіжності до неї.

2.6.2. Геометрія ігрової поверхні

Вибір функції втрат визначає форму ігрової поверхні у просторі (θ_G, θ_D) і, відповідно, характер градієнтної динаміки.

BCE (JSD). При непересічних носіях p_{data} та p_g дивергенція $\text{JSD} = \log 2 = \text{const}$, тому $\nabla_{\theta_G} \mathcal{L} \approx 0$ – поле градієнтів не несе корисного сигналу для генератора.

Wasserstein. Відстань W є ненасиченою: поле градієнтів ненульове скрізь, навіть при непересічних носіях. Gradient penalty додатково регуляризує поверхню, штрафуючи надмірну крутизну критика через доданок $(\|\nabla f_w\|_2 - 1)^2$.

Hinge. Поверхня є кусково-лінійною: у зоні відступу градієнт сталий, поза нею – нульовий. Навколо рівноваги це утворює «плато», яке зупиняє оновлення дискримінатора після досягнення відступу.

2.6.3. Якобіан та власні значення

Лінеаризуючи v навколо θ^* , отримуємо якобіан:

$$J = \begin{pmatrix} -\nabla_{GG}^2 \mathcal{L} & -\nabla_{GD}^2 \mathcal{L} \\ \nabla_{DG}^2 \mathcal{L} & \nabla_{DD}^2 \mathcal{L} \end{pmatrix}$$

Власні значення λ_i матриці J визначають характер рівноваги:

Власні значення	Тип точки	Поведінка
$\text{Re}(\lambda_i) < 0$ для всіх i	Стійкий атрактор	Збіжність до рівноваги
$\text{Re}(\lambda_i) = 0, \text{Im}(\lambda_i) \neq 0$	Центр	Орбіти навколо рівноваги, збіжності немає
$\text{Re}(\lambda_i) > 0$ для деяких i	Нестійка точка	Розбіжність або осциляції

Для BCE-GAN якобіан у рівновазі має суто уявні власні значення (режим центру) [3]. WGAN-GP та SNGAN зміщують спектр до від’ємної дійсної частини, наближаючи динаміку до стійкого атрактора.

2.6.4. Дивергенції як потенціальні функції

Кожна функція втрат неявно визначає **потенціальну функцію** для генератора:

$$\Phi(\theta_G) = D_f(p_{\text{data}} \parallel p_{g(\theta_G)})$$

де D_f – відповідна дивергенція або метрика. Збіжність градієнтного спуску по θ_G залежить від властивостей Φ :

- **BCE:** $\Phi = \text{JSD}$ – стала при непересічних носіях, градієнтний сигнал відсутній.
- **WGAN:** $\Phi = W$ – гладка і строго позитивна скрізь, збіжність гарантована при точному розв’язанні задачі критика.
- **Hinge:** Φ кусково-лінійна – збіжність у зоні відступу, зупинка поза нею.

Таким чином, вибір функції втрат є вибором геометрії ігрового простору і, відповідно, умов існування та стійкості рівноваги Неша.

2.7. Узагальнена теоретико-ігрова модель GAN

2.7.1. Єдина рамка через f-дивергенції

GAN, WGAN та їх варіанти можна уніфікувати через поняття **f-дивергенції**. Для опуклої функції f з $f(1) = 0$ визначають:

$$D_f(p \parallel q) = \int q(x) f\left(\frac{p(x)}{q(x)}\right) dx$$

Класичний GAN відповідає $f(t) = t \log t - (t + 1) \log \frac{t+1}{2}$ (пов’язане з JSD), WGAN – граничному випадку лінійного f . У кожному випадку дискримінатор (або критик) є **варіаційним оцінювачем** дивергенції між p_{data} та p_g .

2.7.2. Сідлова точка та умови першого порядку

Розглянемо загальну мінімаксну задачу та умови, які необхідні для сідлової точки.

Загальна мінімаксна задача:

$$\min_{\theta_G} \max_{\theta_D} \mathcal{L}(\theta_G, \theta_D)$$

Необхідні умови сідлової точки (θ_G^*, θ_D^*) :

$$\nabla_{\theta_D} \mathcal{L} = 0, \quad \nabla_{\theta_G} \mathcal{L} = 0$$

Для нелінійних мереж ці умови не є достатніми (через невиконаність). Саліманс та ін. [4] показали, що **одночасне** градієнтне спускання може осцилювати навколо сідлової точки. Практичні прийоми боротьби з цим:

- **Two time-scale update rule (TTUR)**: різні кроки навчання для G та D.
- **Spectral normalization**: нормування вагових матриць за спектральною нормою гарантує ліпшицевість дискримінатора без явного обрізання.
- **Gradient penalty (WGAN-GP)**: штрафний доданок $\lambda \mathbb{E} \left[(\|\nabla f\|_2 - 1)^2 \right]$ замість weight clipping.

2.7.3. Зв'язок між збіжністю GAN та теоремою мінімаксу

Класична теорема Мінімаксу фон Неймана гарантує $\min \max = \max \min$ для опуклих-угнутих неперервних функцій на компактних множинах. Голцман [2] узагальнив цей результат на нескінченні ігри: мінімаксна рівність зберігається, якщо простори стратегій є польськими просторами і функція виграшу задовольняє умовам вимірності та інтегровності.

У GAN простір параметрів θ_G, θ_D є евклідовим (компактним за наявності weight clipping), але V є невиконуючою–неугнутою по параметрах мереж. Отже, $\min_G \max_D V \neq \max_D \min_G V$ у загальному випадку, і теоретична рівновага може не збігатися із практичною точкою сходження градієнтного спуску. Саме тому дослідження [3], [4], [6] зосереджені на **апроксимації** рівноваги Неша через модифікацію цільової функції, архітектури або оптимізаційного алгоритму.

Підіб'ємо підсумки.

Модель	Цільова функція	Метрика	Стабільність
GAN	Binary cross-entropy	Jensen–Shannon divergence	Низька: vanishing gradients, mode collapse
DCGAN	Binary cross-entropy	Jensen–Shannon divergence	Середня: стабільніше через CNN + BatchNorm
WGAN	Wasserstein distance	Wasserstein-1 distance	Вища: стабільні градієнти, менше mode collapse
WGAN-GP	Wasserstein distance + gradient penalty	Wasserstein-1 distance	Висока: м'яке обмеження Ліпшиця, без деградації ємності
SNGAN	Hinge loss	Margin-based	Висока: spectral normalization гарантує ліпшицевість, стійка рівновага відступу

3. ПРАКТИЧНА ЧАСТИНА

3.1. Програмна реалізація експериментального пайплайну

Для проведення серії порівняльних експериментів з різними функціями втрат та архітектурами розроблено модульний навчальний пайплайн мовою Python 3 з використанням бібліотеки PyTorch. Повний вихідний код доступний у репозиторії: <https://github.com/deymel7/gan-minimax-equilibrium>.

3.1.1. Структура проєкту

Пайплайн побудований за принципом реєстрів (Registry pattern), що дозволяє динамічно підключати різні архітектури, функції втрат та оптимізатори без зміни основного коду навчання.

gan-minimax-equilibrium/

```
├─ configs/          # YAML-конфігурації експериментів
├─ models/           # Генератори та дискримінатори
├─ losses/           # BCE, Wasserstein, Hinge
├─ optimizers/       # фабрика оптимізаторів
├─ transforms/       # пайплайн аугментацій
├─ dataset/          # завантаження Animal Faces
├─ utils/            # Config, Registry, helpers
├─ train.py          # головний цикл навчання
└─ evaluation.ipynb  # аналіз та візуалізація
```

3.1.2. Реєстр компонентів

Центральна абстракція – клас `Registry[T]`, що забезпечує реєстрацію та пошук компонентів за іменем:

```
class Registry(Generic[T]):
    def register(self, name: str) -> Callable[[T], T]:
        def wrapper(obj: T) -> T:
```

```

        self._objects[name] = obj

        return obj

    return wrapper

    def get(self, name: str) -> Optional[T]:
        return self._objects.get(name)

```

Усі моделі ("D", "C", "baseline_G", "G", "WG"), функції втрат ("BCE", "Wasserstein", "Hinge") та оптимізатори реєструються через цей механізм. Конфігурація кожного експерименту описується одним YAML-файлом, що повністю визначає архітектуру, гіперпараметри та тип регуляризації.

3.1.3. Архітектури моделей

Реалізовано три генератори та два дискримінатори. `BaselineGenerator` використовує транспоновані згортки (DCGAN-стиль); `AdvancedGenerator` та `WGANAdvancedGenerator` – блоки `UpsampleResBlock` з `residual`-з'єднаннями та парадигму `Upsample + Conv` замість `ConvTranspose2d`, що пом'якшує шахові артефакти. Дискримінатор `Discriminator` підтримує режими `PatchGAN` і глобального пулінгу, нормалізацію `batch/instance/none` та спектральну нормалізацію. `WGANCritic` – аналог для WGAN з `layer/instance/none` нормалізацією (`BatchNorm` виключено, оскільки порушує умову Ліпшиця).

3.1.4. Цикл навчання та регуляризація

Основний цикл (`train.py`) підтримує:

- чергування кроків дискримінатора та генератора з довільним співвідношенням $n_d : n_g$;
- `gradient penalty` (WGAN-GP) як додатковий доданок до L_D ;
- `weight clipping` для WGAN-WC;
- `instance noise` – адаптивний гаусівський шум, що загасає по епохах;

- обрізання норми градієнтів `clip_grad_norm`;
- логування метрик у WandB та збереження чекпоінтів кожні 10 епох.

3.1.5. Датасет та перетворення

Використано датасет **Animal Faces** [5]: підмножина котячих морд, близько 5 000 зображень, роздільна здатність 512×512 пікселів. Пайплайн перетворень включає зміну розміру, випадкове горизонтальне відображення та нормалізацію до $[-1, 1]$.

3.2. Серія експериментів: Vanilla GAN (BCE-loss)

3.2.1. Експеримент 1. Baseline-генератор (BCE, без регуляризації)

Перший експеримент (`test_vanilla_gan_baseline.yaml`) слугував відправною точкою для розуміння базової динаміки гри. Обрано найпростішу конфігурацію: `BaselineGenerator` з транспонованими згортками, дискримінатор без регуляризації (`norm: instance, sn: False, dropout: 0`), симетричні оптимізатори `AdamW` з однаковим $lr = 0.0002$, $n_g = 4$ оновлення генератора на одне оновлення дискримінатора.

Дискримінатор виявився значно сильнішим за генератор і дуже швидко став впевнено класифікувати реальні та синтетичні зображення ($D(x) \approx 1$, $D(G(z)) \approx 0$). Отримуючи від дискримінатора впевнену відмову, генератор стикався з практично нульовим градієнтом (проблема насичення BCE), що призвело до вибухових градієнтів і колапсу мод. Генератор навчився відтворювати кілька візуально схожих зразків, а не покривати розподіл p_{data} загалом.

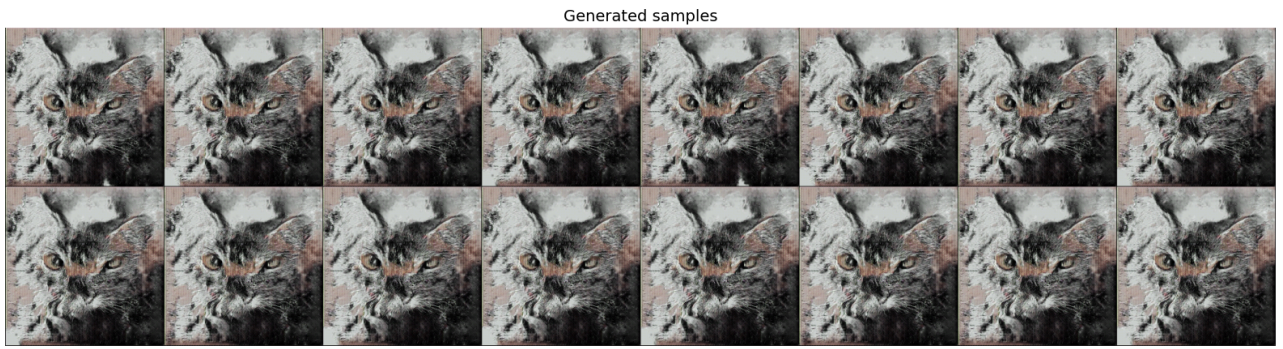


Рисунок 9: Baseline Vanilla GAN: mode collapse – генератор відтворює вузьку підмножину мод розподілу

3.2.2. Експеримент 2. Збалансований Vanilla GAN (BCE + регуляризація D)

На основі висновків попереднього дослідження розроблено збалансовану конфігурацію (`test_vanilla_gan.yaml`): `AdvancedGenerator` з `residual`-блоками, `dropout` 0.4 та менший `lr` дискримінатора (3×10^{-5} проти 5×10^{-4} для генератора), $n_g = 5$, `label_smoothing` = 0.05, норма батча замість `instance-normalization`. Асиметричні темпи навчання є практичним аналогом TTUR (Two Time-Scale Update Rule) [4].

Граденти більше не вибухали, обидва гравці поступово оптимізувались. Однак дискримінатор виявився недостатньо сильним, щоб якісно розрізняти реальні та синтетичні зображення, – градієнтний сигнал для генератора залишався слабким. Обидві моделі навчилися, але якість генерації була низькою.

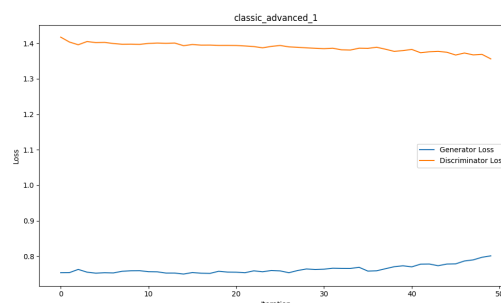


Рисунок 10: Криві втрат: Vanilla GAN зі збалансованими гіперпараметрами

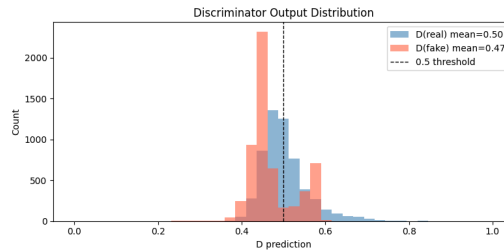


Рисунок 11: Розподіли оцінок $D(x)$ та $D(G(z))$: дискримінатор не впевнений у своїх рішеннях

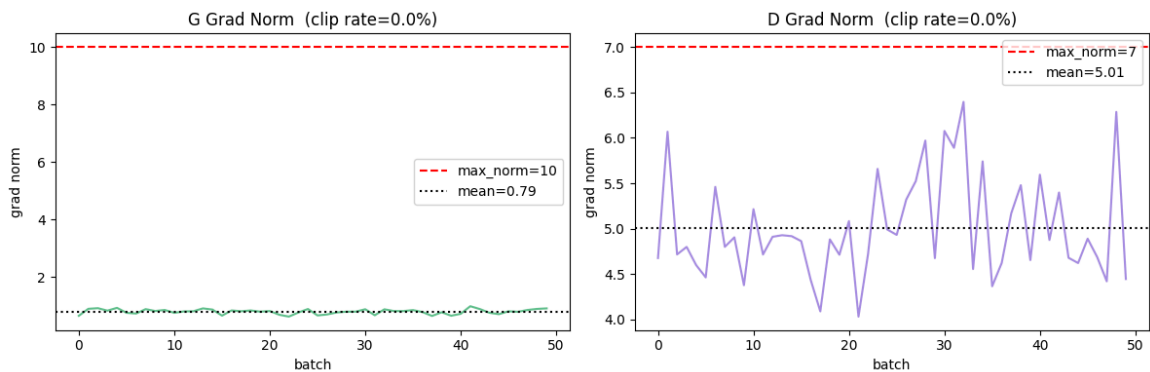


Рисунок 12: Норми градієнтів G та D: стабілізовані, але малоінформативні

Загальний висновок щодо Vanilla GAN. BCE-loss в поєднанні з насиченням логарифму не забезпечує стабільного градієнтного сигналу: або дискримінатор «перемагає» і градієнт зникає, або він надто слабкий і генератор не отримує корисного зворотного зв'язку. Для досягнення кращої рівноваги необхідне використання метрики, що не насичується, – відстані Васерштайна.

3.3. Серія експериментів: Wasserstein GAN

3.3.1. Експеримент 3. WGAN з weight clipping (WGAN-WC)

Конфігурація `test_wgan_wc.yaml`: `WGANAdvancedGenerator` («WG»), `WGANCritic` («C»), `weight_clip = 0.02`, оптимізатор `RMSprop` (`lr = 0.0001`), $n_d = 5$, без нормалізації та без `spectral norm`.

Відстань Васерштайна мала забезпечити стабільний ненасичений градієнтний сигнал порівняно з BCE. `Weight clipping` – примітивний але прямолінійний спосіб накласти умову Ліпшиця. Але, на жаль, вибух градієнтів виявився ката-

строфічним: норми сягали $\sim 250,000,000$ на початку та зменшились лише до $\sim 5,000,000$ без монотонної динаміки. Обрізання ваг до $[-0.02, 0.02]$ призвело до деградації ємності критика: він виводив майже константну відповідь і не міг якісно оцінити різницю між реальними та синтетичними зображеннями. Умова Ліпшиця не виконувалась, хоча саме це є ключовою ціллю weight clipping [6].

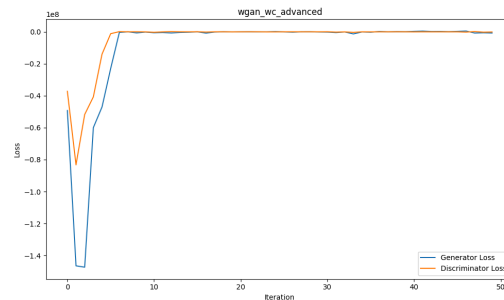


Рисунок 13: WGAN-WC: криві втрат з вибухом градієнтів на початку навчання

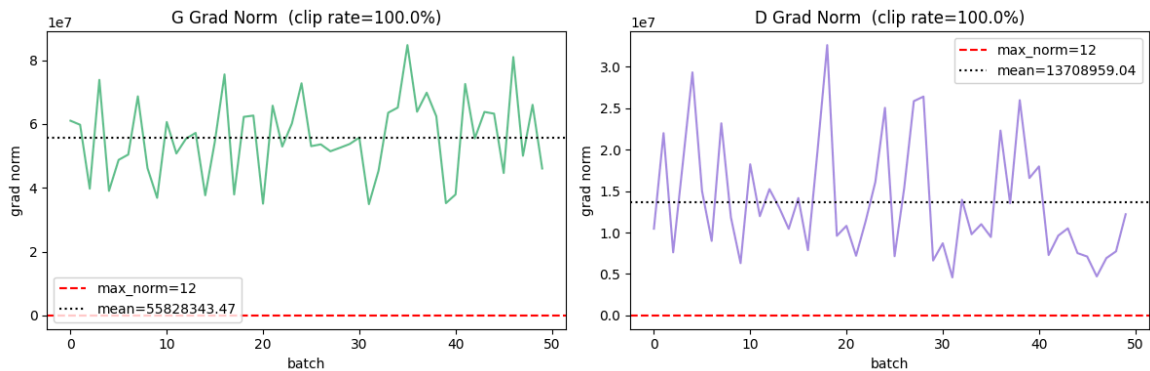


Рисунок 14: Норми градієнтів WGAN-WC: неконтрольоване зростання

3.3.2. Експеримент 4. WGAN зі spectral normalization (WGAN-SN)

Конфігурація `test_wgan_sn.yaml`: та сама пара G/C, але `sn: True`, `weight_clip` відсутній, оптимізатор Adam (betas: $[0., 0.9]$), $n_d = 5$.

Спектральна нормалізація має обмежити спектральну норму кожної матриці ваг критика, наблизивши його до 1-ліпшицевості без явного обрізання. Хоча градієнти зменшились порівняно з WC, але залишались великими. Втрати обох мереж стали від'ємними ($L_G \approx -750$, $L_C \approx -5$), що відповідає оцінці Васерштайна, однак критик так і не навчився якісно оцінювати зображення – вибух

градієнтів зберігся. Причина не відома для мене, спектральна норма використовувалася у всіх шарах WGAN_Critic.

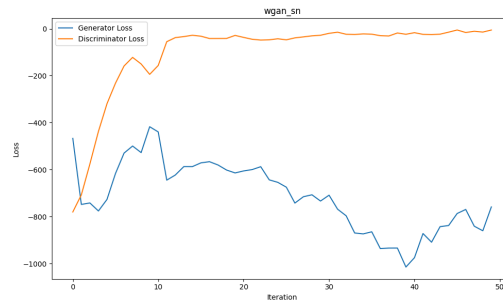


Рисунок 15: WGAN-SN: від’ємні втрати, але нестабільна динаміка

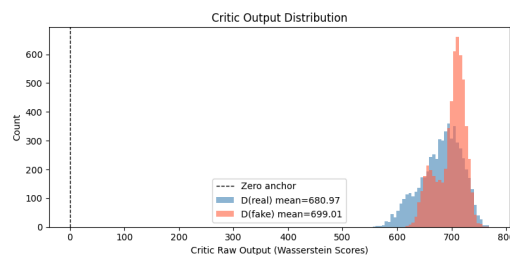


Рисунок 16: Розподіли оцінок критика: G та C не можуть знайти рівновагу

3.3.3. Експеримент 5. WGAN з gradient penalty (WGAN-GP) – найкраща конфігурація

Конфігурація `test_wgan_gp.yaml`: WGANAdvancedGenerator + WGAN_Critic, `grad_penalty_lambda = 10`, `sn: False`, `norm: layer`, `patch: True`, `dropout: 0.3`, $n_d = 8$, Adam з `betas: [0., 0.9]`, `lr_C = 0.0003`, `lr_G = 0.0001`.

Ключове рішення – PatchGAN-голова у критик (patch: True): вона надає просторово-узгоджену оцінку, тобто критик отримує більше градієнтного сигналу про локальну структуру зображення, що підсилює штраф GP і робить ліпшицеву умову ефективнішою.

Градiєнти виявились **ідеальними**: середня норма G – ≈ 1.7 , D – ≈ 5.4 , без вибухів. Втрати залишаються малими (особливо для критика). Через певний час обидві моделі вийшли на якийсь вид рівноваги. Це найкращий результат серед усіх конфігурацій.

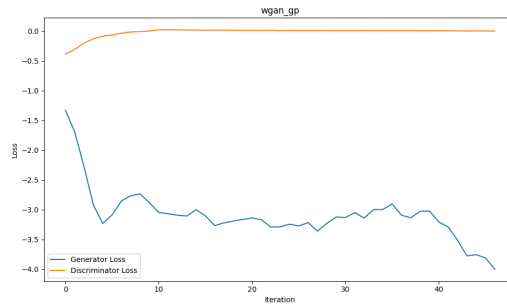


Рисунок 17: WGAN-GP: стабільні криві втрат, рівновага досягається

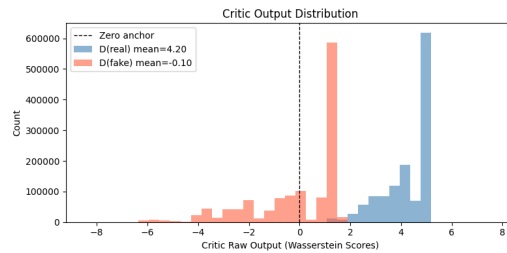


Рисунок 18: Розподіли оцінок критика WGAN-GP: G та C поступово наближа-
ЮТЬСЯ

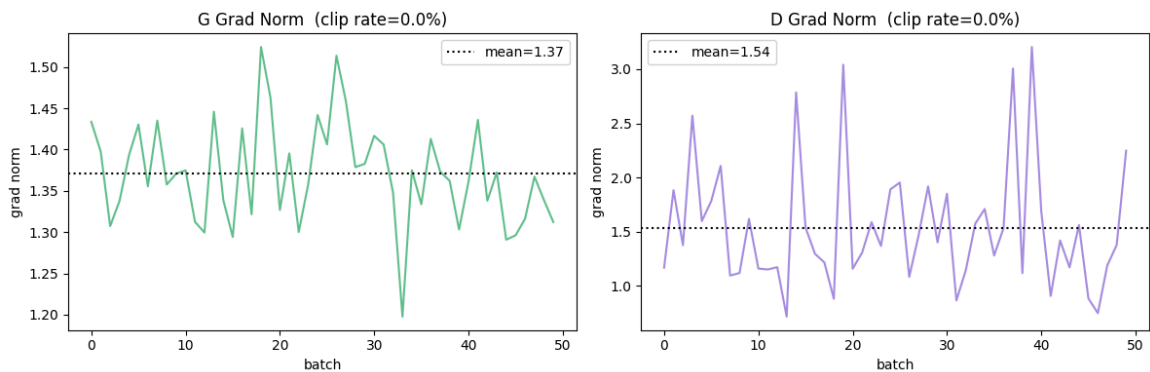


Рисунок 19: Норми градієнтів WGAN-GP: контрольовані та стабільні

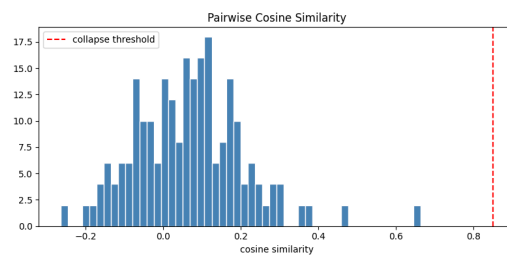


Рисунок 20: WGAN-GP: відсутність mode collapse, різноманітність зразків

3.4. Серія експериментів: Hinge Loss GAN

3.4.1. Експеримент 6. Hinge GAN (базова конфігурація)

Конфігурація `test_hinge_gan.yaml`: `AdvancedGenerator + Discriminator`, `HingeLoss` без `label_smoothing`, `norm: batch`, `dropout: 0.3`, асиметричні `lr` (2×10^{-5} для `D`, 5×10^{-5} для `G`), $n_g = n_d = 1$, відносно великі `G_max_norm = 20`, `D_max_norm = 7`.

`Hinge loss` мала забезпечити чіткіший сигнал через відступ (*pt1*) та зупинити оновлення після досягнення рівноваги. Обидві моделі оптимізувались, і рівновага Неша майже досягалась. Однак навчання виявилось дуже нестабільним: градієнти вибухали, через що генератор не міг стабільно навчатися від дискримінатора. Сам дискримінатор виконував свою функцію, але нестабільність градієнтів не дозволяла `G` стабільно від нього навчатися.

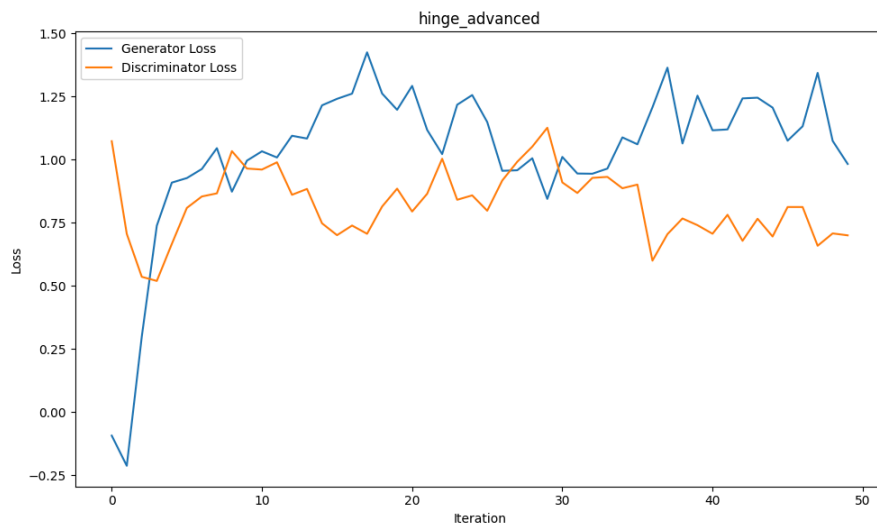


Рисунок 21: Hinge GAN: нестабільне навчання, флуктуючі втрати

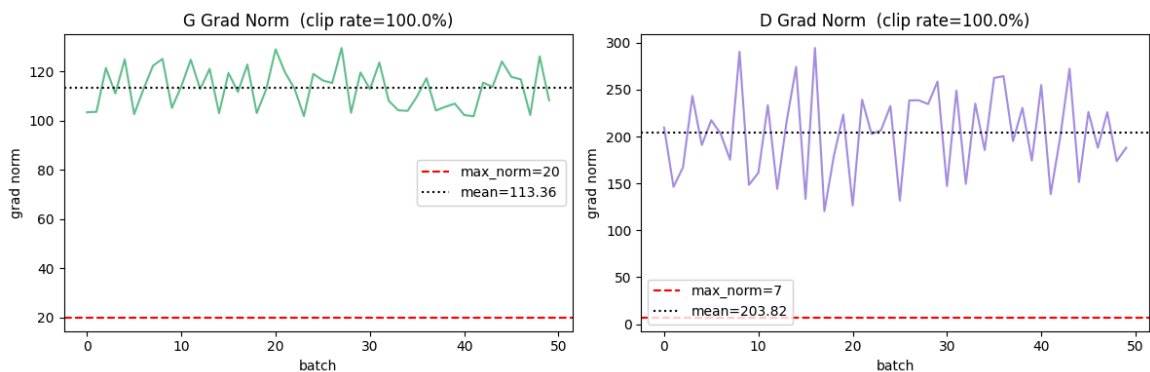


Рисунок 22: Норми градієнтів Hinge GAN: вибухи, особливо у `G`

3.4.2. Експеримент 7. Hinge GAN зі spectral normalization (SNGAN)

Конфігурація `test_hinge_gan_sn.yaml`: `sn: True`, `norm: none`, `dropout: 0.2`, `instance_noise: True`, `label_smoothing = 0.05`, $n_d = 2$, $lr_D = 7 \text{ times } 10^{-5}$, $lr_G = 10^{-4}$.

Spectral normalization у поєднанні з hinge loss – класична архітектура SNGAN [10]. Instance noise поступово загасає і служить додатковою регуляризациєю на ранніх епохах.

SN обмежить спектральну норму ваг дискримінатора, що разом із hinge loss має стабілізувати навчання. Вибухи градієнтів зменшились, але не зникли повністю. Дискримінатор дещо сильніший за генератор – рівновага зміщена. Навчання стабільніше, ніж у базовій конфігурації, але поступається WGAN-GP.

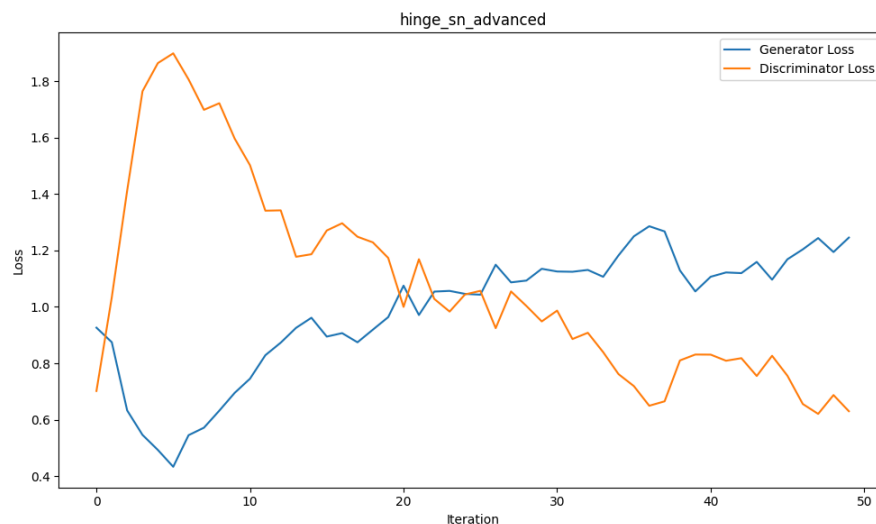


Рисунок 23: SNGAN: порівняно стабільніші криві, але залишаються флуктуації

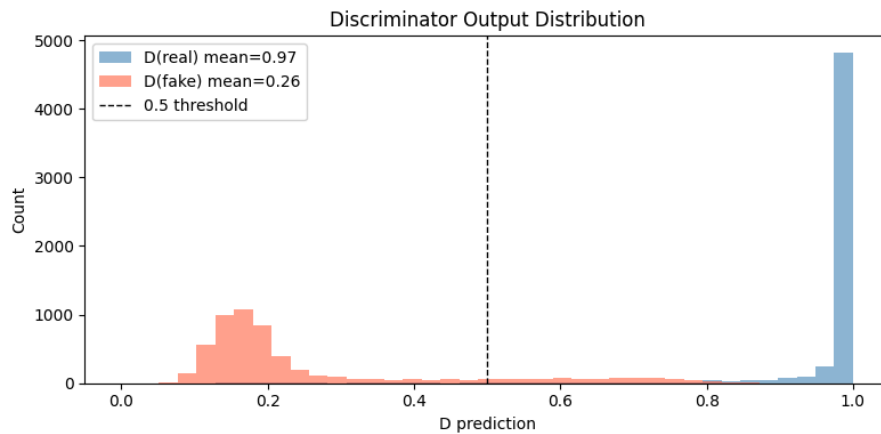


Рисунок 24: Розподіли оцінок SNGAN: D трохи сильніший за G

3.4.3. Експеримент 8. Hinge GAN з gradient penalty (Hinge-GP)

Конфігурація `test_hinge_gan_gp.yaml`: `sn: False`, `norm: none`, `dropout: 0.3`, `grad_penalty_lambda = 10`, `instance_noise: True`, `label_smoothing = 0.05`, $n_d = 2$, `seed = 277`.

Gradient penalty ефективно обмежив вибухи градієнтів (краще, ніж у базовій конфігурації). На старті обидві втрати зростали, потім поступово оптимізувались – певна рівновага досягалась. Однак для повного уникнення вибухів потрібна додаткова регуляризація. Загалом конфігурація збалансованіша за SNGAN, але поступається WGAN-GP за стабільністю.

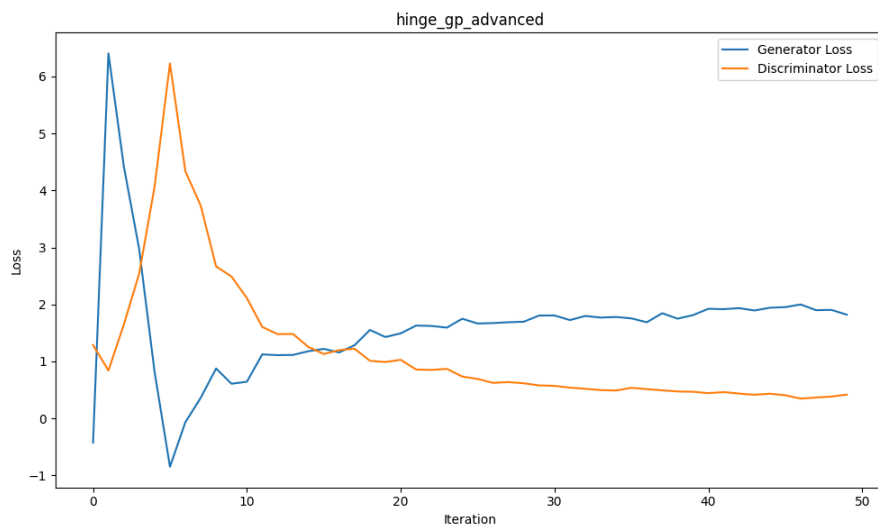


Рисунок 25: Hinge-GP: оптимізація обох мереж із помірними вибухами

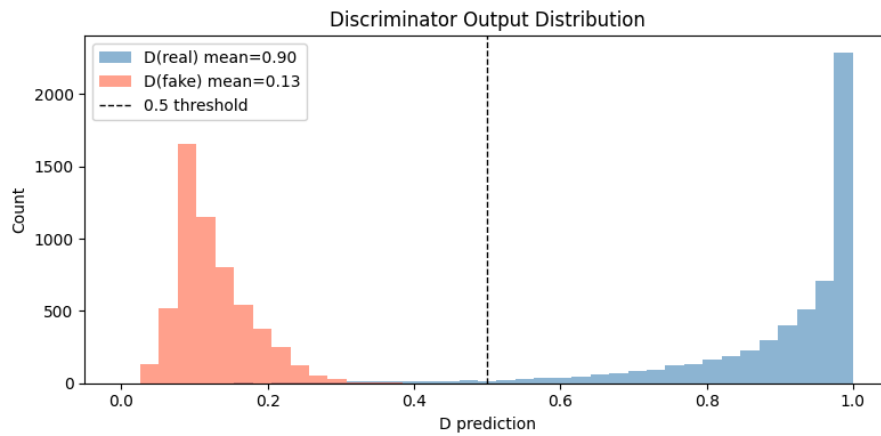


Рисунок 26: Розподіли оцінок Hinge-GP: поступове наближення до рівноваги

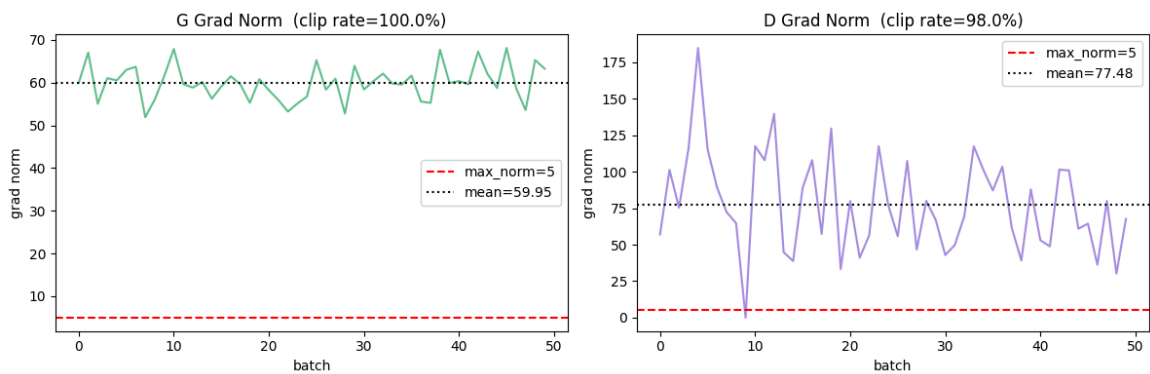


Рисунок 27: Норми градієнтів Hinge-GP: контрольованіші, ніж у SNGAN

3.5. Підсумкові результати генерації

Для якісної оцінки наведено зображення, синтезовані WGAN-GP після всього 50 епох навчання.



Рисунок 28: Зразки, згенеровані WGAN-GP після 50 епох

Незважаючи на обмежену кількість епох (50), зразки WGAN-GP помітні риси котячих морд. Інші конфігурації або виявляли mode collapse (Baseline), або генерували розмиті артефакти (WGAN-WC, WGAN-SN).

4. ВИСНОВКИ

У роботі проведено теоретичний аналіз та практичну перевірку теоретико-ігрових властивостей GAN на задачі генерації зображень котячих морд.

Навчання GAN формалізовано як двоосібну мінімаксну гру, єдина рівновага Неша якої відповідає $p_g = p_{\text{data}}$. Показано, що через невивуклість параметричного простору нейромереж якобіан у точці рівноваги має суто уявні власні значення – це породжує осциляції замість збіжності і пояснює нестабільність навчання на практиці.

Серія з восьми конфігурацій підтвердила, що вибір функції втрат визначає геометрію ігрової поверхні. BCE-loss насичується при непересічних носіях розподілів: базова конфігурація виродилась у mode collapse, а спроба збалансувати гравців через dropout та асиметричні темпи навчання вирівняла втрати, але не покращила якість генерації. Перехід до відстані Васерштайна усунув проблему насичення, однак реалізація умови Ліпшиця виявилась критичною: weight clipping призвів до деградації ємності критика, spectral normalization зменшила, але не усунула вибухи. Найкращий результат продемонструвала конфігурація WGAN-GP з PatchGAN-головою критика: норми градієнтів стабілізувались на рівні ≈ 1.7 (G) і ≈ 5.4 (C), обидві моделі досягли рівноваги, а синтезовані зображення відзначаються найбільшою різноманітністю. Hinge loss зі spectral normalization (SNGAN) стала другою за якістю, поступившись WGAN-GP за стабільністю градієнтів.

Таким чином, вибір функції втрат у GAN – це вибір метрики на просторі розподілів і, відповідно, умов існування та стійкості рівноваги мінімаксної гри.

5. СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- [1] I. J. Goodfellow *et al.*, «Generative Adversarial Nets», *Advances in Neural Information Processing Systems*, вип. 27, 2014.
- [2] R. Holzman, «The Minimax Property in Infinite Two-Person Win-Lose Games», *International Journal of Game Theory*, 1992.
- [3] S. Arora i Y. Zhang, «Do GANs Actually Learn the Distribution? An Empirical Study», *arXiv preprint arXiv:1706.08224*, 2017.
- [4] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, i X. Chen, «Improved Techniques for Training GANs», *Advances in Neural Information Processing Systems*, вип. 29, 2016.
- [5] A. Mvd, «Animal Faces». Kaggle, 2020.
- [6] M. Arjovsky, S. Chintala, i L. Bottou, «Wasserstein GAN», *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- [7] A. Radford, L. Metz, i S. Chintala, «Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks», *arXiv preprint arXiv:1511.06434*, 2015.
- [8] J. Wang i et al., «Improving GAN Equilibrium by Raising Spatial Awareness», *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [9] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, i A. Courville, «Improved Training of Wasserstein GANs», *Advances in Neural Information Processing Systems*, вип. 30, 2017.
- [10] T. Miyato, T. Kataoka, M. Koyama, i Y. Yoshida, «Spectral Normalization for Generative Adversarial Networks», *International Conference on Learning Representations*, 2018.

6. ДОДАТКИ

Компонент підвищення роздільної здатності G

```
class WGANUpsampleResBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.upsample = nn.Upsample(scale_factor=2)
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, padding=1,
bias=False)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1,
bias=False)
        self.activate = nn.ReLU(inplace=True)
        self.identity = nn.Conv2d(in_ch, out_ch, 1) if in_ch !=
out_ch else nn.Identity()

        g0 = min(8, in_ch)
        g1 = min(8, out_ch)
        self.norm0 = nn.GroupNorm(g0, in_ch)
        self.norm1 = nn.GroupNorm(g1, out_ch)
        self.norm2 = nn.GroupNorm(g1, out_ch)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.upsample(x)
        shortcut = self.identity(x)
        x = self.norm0(x)
        x = self.conv1(x)
        x = self.norm1(x)
        x = self.activate(x)
```

```

x = self.conv2(x)
x = self.norm2(x)
return self.activate((x + shortcut) / math.sqrt(2))

```

Гравець-Мінімізатор (Генератор / Generator)

```

class WGANAdvancedGenerator(nn.Module):
    def __init__(self, n_z: int = 128, image_size: int = 512,
                  image_channels: int = 3, base_channels: int = 512,
min_channels: int = 16):
        super().__init__()
        self.base_size = 4
        self.base_channels = base_channels
        n_blocks = int(math.log2(image_size // self.base_size))

        self.input_layer = nn.Linear(n_z, self.base_size *
self.base_size * base_channels)

        blocks = []
        in_ch = base_channels
        for _ in range(n_blocks):
            out_ch = max(in_ch // 2, min_channels)
            blocks.append(WGANUpsampleResBlock(in_ch, out_ch))
            in_ch = out_ch

        self.upsample_blocks = nn.Sequential(*blocks)
        self.out_block = nn.Sequential(nn.Conv2d(in_ch,
image_channels, kernel_size=1), nn.Tanh())

```

```

def forward(self, z: torch.Tensor) -> torch.Tensor:
    z = self.input_layer(z)
    z = z.view(-1, self.base_channels, self.base_size,
self.base_size)
    return self.out_block(self.upsample_blocks(z))

```

Гравець-Максимізатор (Критик / Critic)

```

class WGAN_Critic(nn.Module):
    def __init__(self, image_channels: int = 3, base_channels: int =
64,
                dropout_rate: float = 0.0, sn: bool = True,
                neg_slope: float = 0.2, norm: str = "none", patch:
bool = True):
        super().__init__()
        assert norm in ("instance", "layer", "none"), "Групова
нормалізація шкодить обмеженню Ліпшиця"
        self.patch = patch
        ch = base_channels

        self.features = nn.Sequential(
            self._conv_block(image_channels, ch, sn=sn, norm="none",
neg_slope=neg_slope),
            self._conv_block(ch, ch * 2, sn=sn, norm=norm,
neg_slope=neg_slope),
            self._conv_block(ch * 2, ch * 4, sn=sn, norm=norm,
dropout_rate=dropout_rate, neg_slope=neg_slope),
            self._conv_block(ch * 4, ch * 8, sn=sn, norm=norm,
dropout_rate=dropout_rate, neg_slope=neg_slope),

```

```

        self._conv_block(ch * 8, ch * 8, sn=sn, norm=norm,
dropout_rate=dropout_rate, neg_slope=neg_slope),
    )

    if patch:
        self.head = nn.Conv2d(ch * 8, 1, kernel_size=4,
stride=1, padding=1, bias=True)

        if sn: self.head = spectral_norm(self.head)
    else:
        self.pool = nn.AdaptiveAvgPool2d((4, 4))
        self.head = nn.Sequential(
            nn.Flatten(),
            spectral_norm(nn.Linear(ch * 8 * 4 * 4, ch * 8)) if
sn else nn.Linear(ch * 8 * 4 * 4, ch * 8),
            nn.LeakyReLU(neg_slope),
            nn.Dropout(dropout_rate),
            spectral_norm(nn.Linear(ch * 8, 1)) if sn else
nn.Linear(ch * 8, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        return self.head(x) if self.patch else
self.head(self.pool(x))

    @staticmethod
    def _conv_block(in_c: int, out_c: int, kernel_size: int = 4,
stride: int = 2, padding: int = 1,

```

```

        dropout_rate: float = 0.0, neg_slope: float =
0.2, sn: bool = True, norm: str = "none") -> nn.Sequential:
    conv = nn.Conv2d(in_c, out_c, kernel_size=kernel_size,
stride=stride, padding=padding, bias=(norm == "none"))
    if sn: conv = spectral_norm(conv)
    layers = [conv]
    if norm == "instance":
        layers.append(nn.InstanceNorm2d(out_c, affine=True))
    elif norm == "layer":
        layers.append(_ChanLayerNorm(out_c))
    layers.append(nn.LeakyReLU(neg_slope, inplace=True))
    if dropout_rate > 0.0:
        layers.append(nn.Dropout2d(dropout_rate))
    return nn.Sequential(*layers)

```